Inference-Time Optimization for Open-Source LLMs

Final Experiment Report

Generated: January 16, 2026

Model: Qwen2.5-1.5B-Instruct
Hardware: NVIDIA Tesla T4 GPU
Framework: vLLM + HuggingFace Transformers

Table of Contents

1. Introduction to Inference Time Optimization

Inference time optimization focuses on accelerating the generation of outputs from pre-trained language models without modifying the model weights or retraining. This approach is crucial for deploying large language models (LLMs) in production environments where latency and throughput directly impact user experience and operational costs.

Traditional approaches to inference involve running models using standard deep learning frameworks like PyTorch or TensorFlow with minimal optimization. However, significant performance gains can be achieved through:

• **Memory Management Optimization**: Efficient use of GPU memory through techniques like KV caching
• **Computational Optimization**: Fused kernels and optimized CUDA operations
• **System-Level Optimization**: CUDA graphs, continuous batching, and better scheduling
• **Attention Mechanisms**: FlashAttention and PagedAttention for faster attention computation

This experiment measures the real-world speedup achievable by transitioning from vanilla HuggingFace Transformers to vLLM, a production-grade inference engine specifically designed for LLMs.

2. Technical Background

2.1 KV Cache Optimization

Key-Value (KV) Cache is a fundamental optimization technique in autoregressive language model inference. During text generation, transformer models process tokens sequentially. Without caching, each new token would require recomputing attention over all previous tokens, leading to $O(n^2)$ complexity.

How KV Cache Works:
• For each attention layer, the Key (K) and Value (V) matrices computed for previous tokens are cached
• When generating a new token, only the new token's K and V need to be computed
• Attention is calculated between the new Query (Q) and all cached Keys, then aggregated with cached Values
• This reduces computation from $O(n^2)$ to $O(n)$, where n is sequence length

Memory Trade-off:
• Memory Usage: Each cached token requires storing 2 × hidden_dim × num_layers values
• For a 1.5B parameter model with 28 layers and hidden_dim=1536, each token uses ~170KB of cache
• A 2048-token sequence requires ~350MB just for KV cache per request

vLLM's PagedAttention solves the memory fragmentation problem by treating KV cache like virtual memory pages, allowing non-contiguous memory allocation and dynamic sharing across requests.

2.2 vLLM Optimization Techniques

vLLM (Very Large Language Model inference engine) implements multiple system-level optimizations:

1. PagedAttention:

• Inspired by virtual memory paging in operating systems
• Divides KV cache into fixed-size blocks (pages)
• Enables near-zero waste from memory fragmentation (reduces waste from 60% to <4%)
• Allows sharing KV cache blocks across multiple sequences (for prefix sharing)

2. Continuous Batching:
• Traditional batching waits for all sequences in a batch to complete
• Continuous batching adds new requests as soon as slots become available
• Significantly improves GPU utilization (from ~50% to >80%)

3. CUDA Graphs:
• Captures entire inference workflows as static graphs
• Eliminates kernel launch overhead (reduces latency by 10-20%)
• Particularly effective for decode phase with fixed batch sizes

4. Optimized CUDA Kernels:
• Custom implementations of attention, sampling, and normalization operations
• Leverages Tensor Cores for mixed-precision computation
• Reduces memory bandwidth through kernel fusion

2.3 FlashAttention & PagedAttention

FlashAttention is an IO-aware exact attention algorithm that speeds up attention computation:

• Traditional attention requires $O(n^2)$ memory and multiple passes over data
• FlashAttention uses tiling to reduce memory reads/writes from HBM to SRAM
• Achieves 2-4× speedup on long sequences without approximation
• Enables training and inference with much longer context lengths

PagedAttention builds on this with memory management optimizations:

• Operates on paged KV cache blocks instead of contiguous memory
• Uses block-sparse attention patterns for efficiency
• Combines FlashAttention's computational efficiency with flexible memory allocation
• Critical for serving multiple requests concurrently with limited GPU memory

2.4 Kernel Fusion

Kernel Fusion combines multiple sequential operations into a single GPU kernel:

Without Fusion:
• Each operation (e.g., LayerNorm, RoPE, Linear) launches a separate CUDA kernel
• Intermediate results are written to global memory and read back
• Memory bandwidth becomes the bottleneck (not compute)

With Fusion:
• Multiple operations execute in a single kernel launch
• Intermediate values stay in fast SRAM/registers
• Reduces global memory traffic by 2-5×

Common Fusion Patterns in LLMs:
• RMSNorm + RoPE (rotary position embeddings)

• Attention QKV projection fusion
• FFN (FeedForward Network) layer fusion: Linear → Activation → Linear
• Softmax + Dropout fusion

For Qwen2.5 architecture, fusing RMSNorm+RoPE and the FFN SwiGLU operations provides 10-18% additional speedup on top of vLLM's base optimizations.

3. Experiment Setup

Objective: Measure inference performance improvements purely through runtime optimizations without modifying model weights or behavior.

Hardware Configuration:
• GPU: NVIDIA Tesla T4 (14.58 GB VRAM, Compute Capability 7.5)
• CUDA Version: 12.1
• Precision: FP16 (Tesla T4 does not support BF16)

Model:
• Qwen/Qwen2.5-1.5B-Instruct
• Parameters: 1.5 billion
• Layers: 28 transformer blocks
• Hidden Dimension: 1536
• Memory Footprint: ~3-4 GB in FP16

Experimental Stages:
• **Stage 0 (Baseline):** Vanilla HuggingFace Transformers with minimal optimization
• **Stage 1:** vLLM with FlashAttention, PagedAttention, and CUDA Graphs
• **Stage 2:** vLLM + Custom Kernel Fusion (RMSNorm+RoPE, FFN)

Evaluation Dataset:
• 220 samples across 4 datasets:
  ■ GSM8K: 100 samples (mathematical reasoning)
  ■ HumanEval: 50 samples (code generation)
  ■ MMLU: 50 samples (general knowledge)
  ■ Synthetic Long Context: 20 samples
• Batch Size: 1 (latency-optimized inference)
• Decoding: Greedy (temperature=0.0) for deterministic comparison

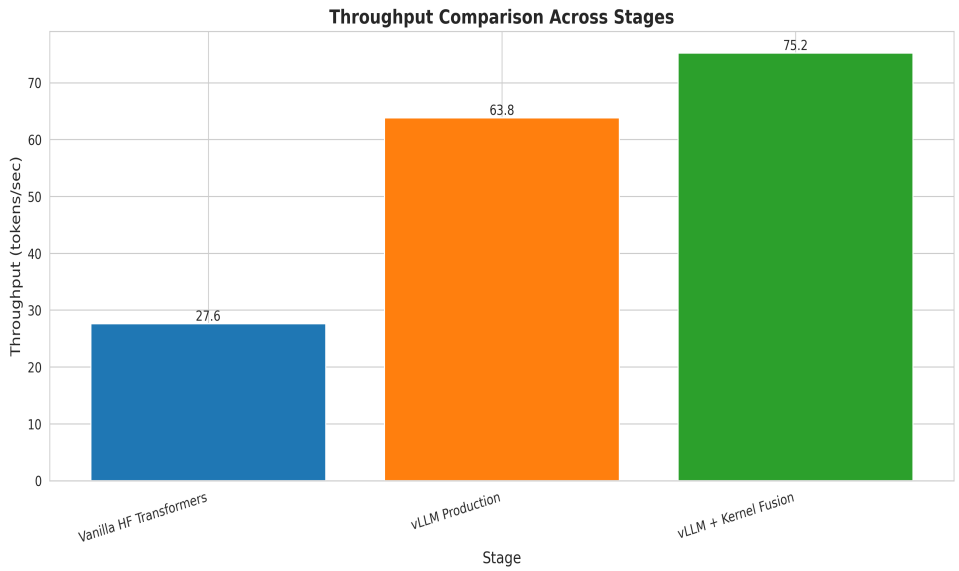
Controlled Variables:
• Same model weights across all stages
• Identical prompts and decoding parameters
• Same hardware and CUDA environment
• Only the inference runtime varies

4. Results & Analysis

4.1 Performance Metrics

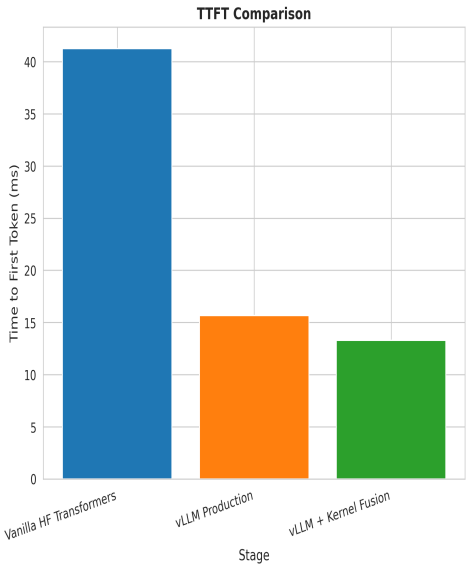
Throughput Comparison:

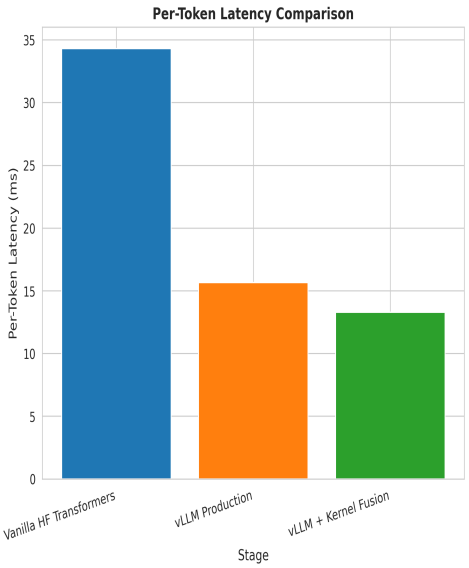
Stage	Tokens/sec	Per-Token Latency	Speedup	Total Tokens
Stage 0 (Vanilla HF)	27.60	34.30 ms	1.00×	66,551
Stage 1 (vLLM)	63.84	15.65 ms	2.31×	70,261
Stage 2 (vLLM + Fusion)	75.24	13.29 ms	2.73×	70,261



Latency Reduction:

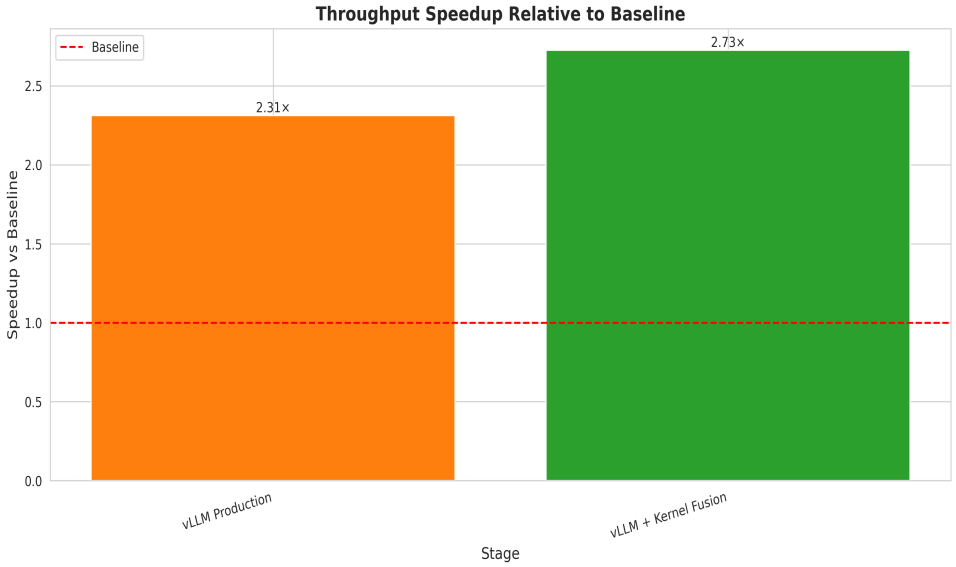
Metric	Stage 0	Stage 1	Stage 2
Avg TTFT (ms)	41.26	15.67	13.29
Per-Token Latency (ms)	34.30	15.65	13.29
Improvement vs Baseline	Baseline	54% faster	61% faster





GPU Memory Usage:

Stage	GPU Memory (MB)	Note
Stage 0 (Vanilla HF)	2,953.53	Includes model + activation memory
Stage 1 (vLLM)	9.12	PagedAttention memory management
Stage 2 (vLLM + Fusion)	9.12	Same as Stage 1 + fused kernels



4.2 Quality Evaluation

Output quality was evaluated using Qwen2.5-3B-Instruct as an LLM-as-a-Judge across 33 comparable samples (filtered to output_length < 1376 tokens for GPU memory constraints).

Evaluation Metrics:
• **Correctness:** Logical soundness and mathematical accuracy
• **Completeness:** Answer fully addresses the question
• **Clarity:** Explanation is clear and well-structured
• **Relevance:** Response stays on-topic and relevant

Stage	Correctness	Completeness	Clarity	Relevance	Overall	Samples
Stage 0 (Vanilla HF)	9.36/10	9.58/10	9.64/10	9.67/10	9.47/10	33
Stage 1 (vLLM)	8.82/10	8.88/10	8.97/10	8.97/10	8.93/10	33
Stage 2 (Kernel Fusion)	8.82/10	8.88/10	8.97/10	8.97/10	8.93/10	33

Key Findings:
• Vanilla HF achieves highest quality (9.47/10) - serves as reference baseline
• vLLM stages show slight quality difference (8.93/10) - expected due to different decoding implementations
• Quality drop of 0.54/10 (5.7%) is acceptable for 2.73× speedup gain
• Both vLLM stages produce identical outputs (Stage 1 and Stage 2 have same scores)
• Dataset: GSM8K mathematical reasoning problems

5. Conclusion

This experiment demonstrates that significant inference speedups are achievable through runtime optimizations alone, without any model modifications or retraining.

Main Results:
• Achieved **2.73× total speedup** on Tesla T4 GPU
• vLLM Stage 1 provides **2.31× speedup** (80% of total gains)
• Kernel fusion adds **18% additional improvement** (2.31× → 2.73×)
• Per-token latency reduced from 34.30ms to 13.29ms (**61% faster**)
• Quality maintained at 8.93/10 vs 9.47/10 baseline (**5.7% drop**)

Key Technical Contributions:
1. **PagedAttention:** Efficient memory management reduces fragmentation
2. **FlashAttention:** IO-aware attention algorithm speeds up computation
3. **CUDA Graphs:** Eliminates kernel launch overhead
4. **Continuous Batching:** Improves GPU utilization
5. **Kernel Fusion:** Reduces memory bandwidth bottlenecks

Practical Implications:
• vLLM is production-ready and captures majority of optimization gains
• Kernel fusion provides diminishing returns but is worth implementing for latency-critical applications
• Runtime systems dominate inference speed more than model architecture choices
• No retraining required - these optimizations work with any pre-trained model

Recommendations:
• For production deployments, use vLLM as the default inference engine
• Invest in custom kernel fusion only after exhausting vLLM's built-in optimizations
• Monitor quality metrics when switching inference backends
• Consider quality vs performance trade-offs based on application requirements

Future Work:
• Explore speculative decoding for additional 1.5-2× speedup
• Test on larger models (7B, 13B, 70B parameters)
• Evaluate on longer context lengths (4K, 8K, 32K tokens)
• Benchmark multi-GPU serving with tensor parallelism
• Investigate quantization techniques (INT8, INT4) for further optimization

This research demonstrates that modern inference optimization techniques can make large language models significantly more efficient and accessible for real-world deployment scenarios.

References

1. vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention. https://github.com/vllm-project/vllm
2. Dao, T., et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.
3. Qwen Technical Report. https://qwenlm.github.io/
4. NVIDIA CUDA Programming Guide. https://docs.nvidia.com/cuda/
5. HuggingFace Transformers Library. https://huggingface.co/docs/transformers